PASSWORD STRENGTH CHECKER

MINI PROJECT REPORT

1. TITLE OF THE PROJECT

Password Strength Checker Using Python

2. ABSTRACT

The Password Strength Checker is a Python-based mini project that checks the strength of a password entered by the user. The program evaluates the password based on five conditions: minimum length, lowercase letters, uppercase letters, numbers, and special characters.

The program classifies passwords as Very Weak, Weak, Medium, Strong, or Very Strong. If the password is Weak or Medium, the program asks the user to enter another password. The program continues until the user enters a Strong or Very Strong password.

This project helps users understand the importance of creating strong passwords for better security.

3. AIM

To develop a Python program that checks password strength and repeatedly asks the user to enter a stronger password until a Strong or Very Strong password is provided.

4. OBJECTIVES

To check the length of a password.

To identify lowercase and uppercase letters.

To check whether the password contains numbers.

To identify special characters.

To classify the password strength.

To repeatedly request a password until it meets the required strength.

5. SOFTWARE REQUIREMENTS

Programming Language: Python

Code Editor: Visual Studio Code

Module Used: `re` (Regular Expression)

Operating System: Windows

6. MODULE DESCRIPTION

Module 1: Password Input

The program receives a password from the user using the `input()` function.

Module 2: Password Validation

The program checks whether the password contains at least eight characters, lowercase letters, uppercase letters, numbers, and special characters.

Module 3: Strength Calculation

The program increases the strength score by one for every condition satisfied.

Module 4: Strength Classification

The password is classified as Very Weak, Weak, Medium, Strong, or Very Strong based on its score.

Module 5: Loop Control

The `while` loop repeatedly asks the user for a password until a Strong or Very Strong password is entered. The `break` statement terminates the loop when an acceptable password is provided.

7. ALGORITHM

Start the program.

Import the `re` module.

Define a function to check password strength.

Initialize the strength score to zero.

Check the password length, lowercase letters, uppercase letters, numbers, and special characters.

Increase the score for every condition satisfied.

Display the password strength based on the score.

If the password is Strong or Very Strong, accept it and terminate the loop; otherwise, ask the user to enter another password.

Stop the program.

8. PYTHON PROGRAM

```
import re

def check_password_strength(password):
    strength = 0

    if len(password) >= 8:
        strength += 1

    if re.search('[a-z]', password):
        strength += 1
```

```
    if re.search('[A-Z]', password):
        strength += 1

    if re.search('[0-9]', password):
        strength += 1

    if re.search('[@!#$%^&*+=-]', password):
        strength += 1

    return strength


def main():
    while True:
        password = input('Enter a password: ')

        strength = check_password_strength(password)

        if strength == 5:
            print('Password strength: Very Strong')
            print('Strong password accepted!')
            break

        elif strength == 4:
            print('Password strength: Strong')
            print('Strong password accepted!')
            break

        elif strength == 3:
            print('Password strength: Medium')
            print('Please enter a stronger password.')

        elif strength == 2:
            print('Password strength: Weak')
            print('Please enter a stronger password.')

        else:
            print('Password strength: Very Weak')
            print('Please enter a stronger password.')


if __name__ == '__main__':
    main()
```

9. SAMPLE OUTPUT

Example 1: Very Weak Password

Input:

```
Enter a password: hello
```

Output:

```
Password strength: Very Weak
Please enter a stronger password.
```

Example 2: Medium Password

Input:

```
Enter a password: hello123
```

Output:

```
Password strength: Medium
Please enter a stronger password.
```

Example 3: Very Strong Password

Input:

```
Enter a password: Hello@123
```

Output:

```
Password strength: Very Strong
Strong password accepted!
```

10. EXPLANATION OF THE PROGRAM

Importing the Module

The `re` module is imported to search for letters, numbers, and special characters in the password.

Defining the Function

The `check_password_strength()` function evaluates the password and calculates its strength score.

Checking Password Length

The condition `len(password) >= 8` checks whether the password contains at least eight characters.

Checking Character Types

The `re.search()` function checks for lowercase letters, uppercase letters, numbers, and special characters.

Using the While Loop

The `while True` statement allows the program to repeat continuously until an acceptable password is entered.

Using the Break Statement

The `break` statement terminates the loop when the password score reaches four or five.

Displaying the Result

The program displays the password strength and provides instructions to the user.

11. ADVANTAGES

Simple and easy to use.

Helps users understand password strength.

Encourages the use of stronger passwords.

Uses Python functions and regular expressions.

Repeats password validation automatically.

Demonstrates the practical use of loops and conditional statements.

12. LIMITATIONS

The program checks only five basic password conditions.

It does not check whether the password has been exposed in a data breach.

It does not measure resistance to advanced password-guessing techniques.

A password classified as Strong is not necessarily secure in every situation.

13. FUTURE ENHANCEMENTS

Add a minimum password length of twelve characters.

Display individual requirements that the password has not satisfied.

Add a graphical user interface.

Add an option to show or hide the password.

Check passwords against known compromised-password databases using a privacy-preserving method.

14. CONCLUSION

The Password Strength Checker project was developed using Python to evaluate password strength based on five basic conditions. It uses functions, loops, conditional statements, and regular expressions to validate passwords.

The program continues asking the user for a password until a Strong or Very Strong password is entered. This project provides practical knowledge of Python programming and demonstrates the importance of creating strong passwords.

15. RESULT

Thus, the Password Strength Checker was successfully implemented using Python. The program identifies password strength and accepts the password only when its score is Strong or Very Strong.